facecam

Camera → motion → local vision LLM → recording + alert · White paper · October 2026

1. Summary

facecam is a Python security-camera system that runs entirely on the owner's own computer. A camera is watched continuously with cheap motion detection. Only when something moves is a single image sent to a **local vision-language model (LLM)**, which says whether a person is in the picture and how sure it is. If a person is confirmed, the Python program saves a video recording and sends a notification to the owner. No image or video leaves the machine except the alert the owner chooses to receive.

2. The whole process



Step	What happens
1. Camera	OpenCV (`cv2.VideoCapture`) reads frames from the webcam or an IP camera. A small rolling buffer keeps the last few seconds in memory, so the recording can include the moments *before* motion was noticed.
2. Motion detection	Each frame is shrunk, blurred and compared with a background model (OpenCV background subtraction). If enough pixels change, motion is declared. This step is very cheap, so it runs on every frame and keeps the LLM idle most of the time. A cooldown (for example 5–10 seconds) stops one event from triggering many LLM calls.
3. Snapshot	When motion starts, the sharpest recent frame is selected and saved as a JPEG, resized to the model's preferred input size to keep inference fast.
4. Local LLM	The image is sent to a vision model running on the same computer (served by Ollama on `localhost`). The prompt asks for strict JSON only: `{"person": true/false, "confidence": 0.0–1.0, "description": "..."}`. The reply is the **accuracy ratio** the system acts on.
5. Decision	Python parses the JSON. If `person` is true and `confidence` is at or above a set threshold (default 0.6), the event is confirmed. Malformed or timed-out replies are treated as "no decision" and logged, never as an alert.
6. Record and notify	On confirmation, Python writes the buffered pre-roll plus live frames to an MP4 (`cv2.VideoWriter`) until motion has stopped for a few seconds. It then sends a notification (phone push via ntfy or Telegram, or email) containing the snapshot, the model's description and its confidence, and the path of the recording.

3. Choosing the local LLM

The goal is the best person-recognition quality with the fastest response, on ordinary hardware. The recommended setup uses two tiers, both small enough to run on a laptop through Ollama:

Role	Model (suggested)	Why
Default	Qwen2.5-VL 3B	Strong at following the "reply in JSON" instruction and at describing people in a scene, at a size that stays responsive on CPU or a modest GPU.
Fastest fallback	Moondream 2	Around 2B parameters, very quick, suited to weak machines; less reliable on small or partly hidden people.
Higher accuracy	Qwen2.5-VL 7B	Better on dark or cluttered scenes if a GPU is available; slower.

This ranking is a starting recommendation, not a measured result. The right choice depends on the user's hardware, so the project should include a small benchmark (see Test plan) that times each model and scores it on the user's own camera images before one is fixed as the default.

4. Why a local LLM instead of a cloud API

Reason	Local LLM	Cloud API (e.g. Claude API)
Privacy	Images stay on the owner's machine and are never handed to a third party.	Every snapshot is uploaded to the provider's servers.
Cost	No per-image fee after the hardware is bought.	Billed per image, which adds up quickly on a busy camera.
Response time	No upload or network round trip; speed depends on the local hardware.	Adds upload time and network delay to every check.
Availability	Works with the internet down, when a camera is needed most.	Stops working without a connection or during an outage.
Control	The owner chooses the model, version and settings, and nothing changes unannounced.	Model, limits and pricing are set by the provider.

Trade-offs. Small local models are generally less accurate than the largest cloud models, and local speed depends on the computer; this is why the benchmark in section 6 matters. Local also does not mean automatically secure: recordings sit on the owner's disk and need encryption and automatic deletion of old clips. Notification services are still third parties, so the owner can choose text-only alerts and keep snapshots and video on the machine.

5. Why this combination: a Python library plus a local LLM

Neither half is enough alone, so facecam splits the job between them:

Approach	Strengths	Weaknesses
Python library only (OpenCV motion detection)	Very fast, tiny CPU use, runs on every frame.	Cannot tell a person from a pet, shadow, curtain or headlights, so it produces many false alarms.
Python library with a trained detector (e.g. YOLO)	Fast and good at the narrow question “is there a person?”.	Gives a label and box, not an understanding of the scene; new needs (“someone carrying a box”) mean retraining or new models.
LLM only (every frame)	Understands the scene and can explain it in words.	Far too slow and power-hungry to run on a continuous video stream.
Library + local LLM (chosen)	The library does what it is best at: continuous, cheap watching, recording and orchestration. The LLM looks only at the few frames that matter, and returns a confidence plus a plain-language description that goes into the alert. Behaviour changes by editing the prompt, not by retraining.	More moving parts; the LLM is slower than a detector on the single frame it checks, and its confidence is self-reported.

If testing shows that only a yes/no person answer is needed, a dedicated detector such as YOLO could replace or back up the LLM. The benchmark is designed to compare them.

6. Hardware needed

The local model is served by **Ollama**, a free tool that runs open models on the owner's computer and exposes them on `localhost`. Sizes below are approximate and should be checked against the models actually installed.

Item	Minimum	Recommended
Computer	Modern laptop or mini PC, 4 cores, 8 GB RAM, running Moondream 2 (slow but usable).	16 GB RAM and a GPU with 6–8 GB of memory (or an Apple-silicon Mac) for Qwen2.5-VL 3B or 7B.
Camera	USB webcam, 720p.	1080p IP camera (RTSP) with infrared night vision, mounted with a clear view of the entrance.
Storage	Tens of GB for models and clips.	An encrypted SSD or drive of 256 GB or more; video is recorded only on confirmed events, so use is far below continuous recording.

Network	None for detection; internet only to send alerts.	A wired link to IP cameras for reliability.
Always-on	The computer must stay powered on and awake.	A small dedicated machine, ideally on a UPS.

7. Design details that make it reliable

- **Confidence is self-reported.** A language model does not produce a calibrated probability; the number is its own estimate. It is useful for ranking and thresholding but should be tuned against real events, and treated as a guide rather than a guarantee.
- **False alarms.** Pets, shadows and headlights cause motion without a person; the LLM step rejects these, and the threshold sets the balance between missed people and false alerts.
- **Optional face check.** The existing facecam modules (YuNet detection, SFace recognition) can run on the confirmed snapshot to tell the owner from a stranger, so the owner is not alerted about themselves.
- **Timeouts.** If the LLM is too slow, fall back to the faster model or send an "unverified motion" alert, so no event is silently lost.

8. Test plan

The goal is evidence for the model choice and the confidence threshold, not impressions.

Test	Method	Measure
Accuracy	Collect a few hundred labelled snapshots from the real camera (person / no person) and run each candidate model over them.	Missed-person rate, false-alert rate, and accuracy at each confidence threshold.
Scenarios	Day, night, rain, backlight, headlights, pets, a person far away, partly hidden or crouching, a person who is the owner.	Results per scenario, to find the weak spots.
Speed	Time each step for each model on the target computer.	Motion-to-alert time (median and slowest 5%), per-step breakdown.
Comparison	Run the same images through a YOLO detector.	Whether the LLM beats it enough to justify the extra cost.
Reliability	24-hour soak test; unplug the camera; stop Ollama; cut the internet; send malformed model replies.	No crashes, no lost events, correct fallback behaviour.
Privacy	Watch outgoing network traffic during a run.	Only the configured notification leaves the machine.

Pass/fail targets (for example the highest acceptable missed-person rate and slowest acceptable alert time) are set by the owner before testing begins, then the threshold is tuned and the tests repeated.

9. Proposed code layout

```
facecam/
  camera.py     read frames, rolling pre-roll buffer
  motion.py     background subtraction, cooldown
  vision.py     send snapshot to local LLM, parse JSON reply
  recorder.py / notify.py   MP4 clips, alerts
  pipeline.py   ties the steps together (main loop)
  cli.py / config.py   commands, thresholds, model, settings
```

A `benchmark` command would run each candidate model over a folder of labelled images (person / no person) and report response time and the false-alert and missed-person rates, giving evidence for the model choice and threshold.

10. Privacy, security and limits

- **Local first.** Frames, snapshots, recordings and LLM inference all stay on the machine. The only outbound data is the notification the owner configures; an alert can be sent as text only if the owner prefers.
- **Sensitive data.** Recordings and face data are personal data. Use an encrypted disk, restrict access, delete old clips automatically, and only film where the law and consent allow.

- **Not a guarantee.** A vision model can miss a person or be fooled. Treat facecam as an extra layer, not the only protection for anything valuable. Its accuracy has not yet been benchmarked.

11. Status

The repository currently contains face detection and recognition. The pipeline above is the design for the next stage, built in this order: motion and snapshot, LLM call, recording, notifications, benchmark, optional face check.